A Project Report

on

Text File Compression using Huffman Coding

*carried out as part of the **Design and Analysis of Algorithms (INT3101)** Submitted*

by

Abhijay Shrivastava
2430030333

Aarush Dayal
2430030303

Section: IT 5 E

in partial fulfilment for the award of the degree of

Bachelor of Technology

in

Information Technology



MANIPAL UNIVERSITY
JAIPUR

Department of Information Technology

MANIPAL UNIVERSITY JAIPUR
RAJASTHAN, INDIA

Oct 2026

1. Introduction

Every character in a normal text file is stored using 8 bits, whether it is a very common letter like 'e' or a rare symbol like '#'. This is wasteful, because a file in which some characters appear far more often than others can be written in fewer bits if the common characters get short codes and the rare ones get long codes. Huffman coding, proposed by David Huffman in 1952, is a greedy algorithm that builds the best such code for a given file.

In this project we wrote a C program that compresses a text file with Huffman coding. It reads the file, counts every character, builds the Huffman tree, finds the code of each character, replaces the text with these codes and then decompresses the result again to check that the original text comes back. Finally it prints a table of codes and a comparison of the sizes before and after.

The project is an application of the greedy method from the course. It also gives a good chance to look at the running time of an algorithm, since the program uses simple arrays and loops (no heap or library containers), so the cost of every step can be counted directly from the code.

2. Formal Problem Description

2.1 Input

A text file of n characters, c1, c2, ..., cn, taken from an alphabet of at most 256 different byte values. The file is given to the program through standard input (./huffman < input.txt). The program accepts files of up to 100,000 characters.

2.2 Output

A binary code for every distinct character that appears in the file, the compressed bit string, and a report with the number of characters, the number of unique characters, the original and compressed sizes, the percentage of space saved and the result of the decompression check.

2.3 Mathematical formulation

Let f(x) be the number of times character x appears in the file and let L(x) be the length of its code. We want a prefix code, that is, no code is the beginning of another code, so that a stream of bits can be decoded without any separator. The total size of the compressed file in bits is

$$B = \Sigma\, f(x) \cdot L(x)$$

and the problem is to find the prefix code that makes B as small as possible. Huffman proved that the code obtained by the following greedy rule is optimal: repeatedly take the two nodes with the smallest frequencies, join them under a new parent whose frequency is their sum, and continue until one node (the root) is left. Walking from the root to a character,

writing 0 for a left branch and 1 for a right branch, gives its code. Because the characters are only at the leaves, no code can be the prefix of another.

2.4 Objective and performance measures

- Space saved: 100 × (original bits − compressed bits) / original bits, where the original is 8 bits per character.
- Average bits per character: B / n, to be compared with 8.
- Correctness: decompressing the bits with the tree must return exactly the original text.
- Size with table: a decoder also needs the table of characters and counts, which we count as 5 bytes per unique character (1 byte for the character and 4 for its count).

2.5 Assumptions and scope

- Input is read as bytes, so any file works, but the program is meant for plain text.
- The compressed data is kept in memory as an array of ints (one bit per place) to keep the code simple. A real compressor would pack 8 bits into one byte and write the result to a file.
- The file size is limited to 100,000 characters and the compressed size to 1,000,000 bits. The program prints a message and stops if the limit is crossed.

3. Methodology

3.1 Program structure

The program is a single C file, huffman.c (about 210 lines), and works in six steps in main():

Step	What it does	Time
1. Read	Reads the file with getchar(), stores it in text[] and counts each character in freq[]	O(n)
2. Build tree	Joins the two smallest unused nodes, U − 1 times	$O(U^2)$
3. Codes	Goes from each character up to the root using parent[], then reverses the path	O(U·h)
4. Compress	Copies the code of every character of the file into bits[]	O(B)
5. Decompress	Follows bits from the root to a leaf, compares with text[]	O(B)
6. Report	Sorts characters by count (bubble sort) and prints the tables	$O(U^2)$

Here n is the number of characters, U the number of unique characters (at most 256), h the height of the tree and B the number of compressed bits.

3.2 Building the tree

The tree is stored in arrays. Nodes 0 to 255 are the leaves (one for each possible character) and new internal nodes are created from 256 onwards. For every node the arrays left[], right[], parent[], isright[] and done[] store its children, its parent, whether it is a right child and whether it has already been joined. Each round of the loop scans all nodes, picks the smallest and the second smallest unused one, and creates a new node with the sum of their counts. After U − 1 rounds, one node is left and that is the root.

We use a plain scan to find the two smallest nodes instead of a min-heap. Each round costs O(U) so the tree takes $O(U^2)$ time in total. With a heap this would be O(U log U), but since U is at most 256 the difference is tiny compared to the time for reading and encoding the file.

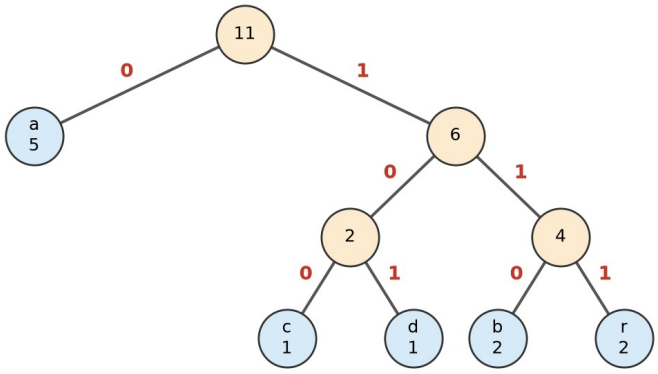


Figure 1: Huffman tree for "abracadabra" as built by the program (0 = left, 1 = right)

3.3 Finding codes, compressing and checking

To get the code of a character, the program starts at its leaf and goes up to the root using parent[], writing '1' if the node is a right child and '0' otherwise. The bits come out backwards, so they are reversed. If the file contains only one kind of character the tree has no edges, so that character is given the code '0'.

Compression then copies the code of each character of the file into bits[] one after the other. For the check, the program starts at the root and for every bit goes to the left or right child. When a leaf (node number below 256) is reached, that character is compared with the next character of the original text and the program returns to the root. The check passes only if every character matches and exactly n characters are decoded.

4. Results

4.1 A small example

For the word "abracadabra" (11 characters) the program gives the following codes:

Char	Count	Code
a	5	0
b	2	110
r	2	111
c	1	100
d	1	101

The most frequent letter, a, gets a 1-bit code and the rare ones get 3 bits. The compressed text is 5×1 + 2×3 + 2×3 + 1×3 + 1×3 = 23 bits instead of 88, which saves 73.86 % (2.09 bits per character). The tree is the one in Figure 1.

4.2 Different kinds of files

We ran the program on five files with different character distributions. The decompression check passed on all of them.

File	Chars	Unique	Compressed (bytes)	Bits / char	Space saved
abracadabra	11	5	3	2.09	73.86 %
English paragraph	198	25	101	4.05	49.43 %
huffman.c (C source)	6537	84	3583	4.38	45.19 %
"ab" repeated 400 times	800	2	100	1.00	87.50 %
Random printable text	5000	94	4118	6.59	17.65 %

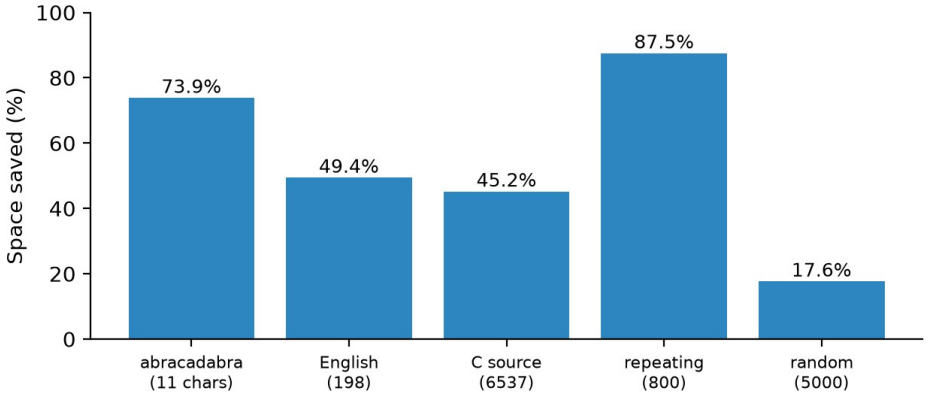


Figure 2: Space saved on the five test files

The saving depends entirely on how uneven the character counts are. The English text and the C source, where spaces and a few letters dominate, shrink by about half. The file made of only two letters needs just 1 bit per character. The random text uses 94 symbols with

almost equal counts, so there is little for Huffman coding to exploit and the saving drops to 17.65 %, close to the limit of log2(94) ≈ 6.55 bits per character.

4.3 The cost of the table

The compressed bits alone are not enough to get the text back, the tree (or the table of counts) must be stored too. With the 5 bytes per unique character we assumed, the real size of small files grows: the English paragraph becomes 101 + 125 = 226 bytes, which is larger than the 198 bytes we started with, and abracadabra becomes 28 bytes instead of 11. The program prints a note when this happens. For the 6,537 character C file, the total is 3,583 + 420 = 4,003 bytes, still a saving of about 39 %. Huffman coding therefore only pays off for files that are long compared to their alphabet.

5. Limitations and future work

- The compressed data is stored as one int per bit and nothing is written to a file, so the program shows the saving but does not create a real .huff file. Packing bits into bytes and writing the tree into the output would make it a complete compressor and decompressor.
- Codes are stored in char code[256][40], so a code can be at most 39 bits long. That is enough for normal text, but a very skewed file with many unique symbols could in theory produce a longer code.
- When the file has only one kind of character, the decompression check is skipped (the program still prints OK). This case should be reported separately.
- The two smallest nodes are found by a linear scan. Using a min-heap would give O(U log U) tree building.
- Files are limited to 100,000 characters. Dynamic arrays and a reading loop in blocks would remove this limit.

6. Tools and how to run

C language, compiled with gcc. Compile and run from the project folder:

```
gcc huffman.c -o huffman   |   ./huffman < input.txt
```

7. References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009 (Chapter 16: Greedy Algorithms).
- D. A. Huffman, "A method for the construction of minimum-redundancy codes", Proceedings of the IRE, 40(9), 1952.
- S. Dasgupta, C. Papadimitriou and U. Vazirani, Algorithms, McGraw Hill, 2008.
- D. Salomon, Data Compression: The Complete Reference, 4th ed., Springer, 2007.